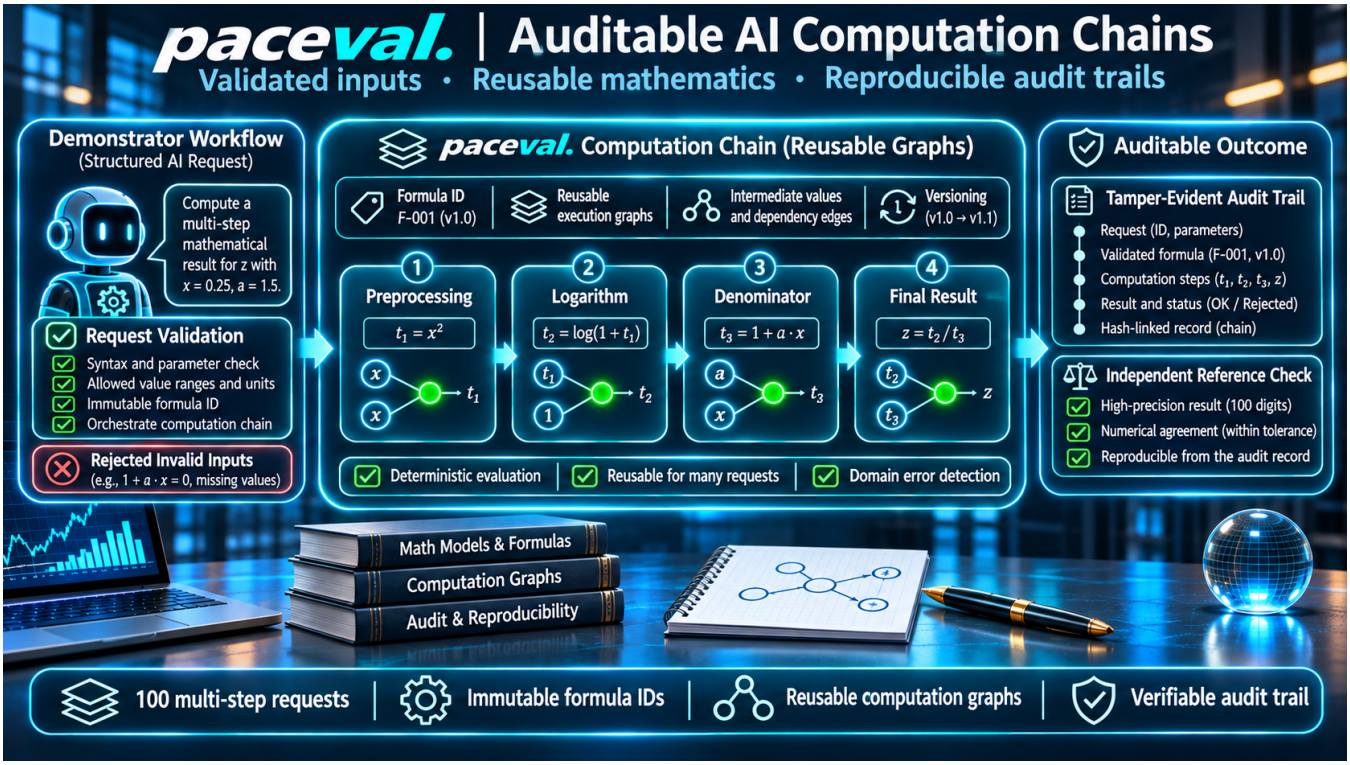
Auditable Multi-Step AI Computation Chains

Direct paceval C/C++ integration | validated requests | immutable mathematical graph identities | independent audit



Illustrative architecture image (marketing artwork, not a captured LLM execution or numerical test).

Implemented workload	Mathematical verification	Audit integrity
110 accepted + 20 rejected requests	440/440 steps within 100-digit reference tolerance	680/680 SHA-256 records verified

Research result

A reusable four-step mathematical computation chain was implemented in C++ with the original paceval Linux runtime. Versioned formula identities, intermediate results, finite-domain rules and a hash-linked trace were independently checked. This is evidence for the specified synthetic offline use case, not evidence of a deployed LLM/MCP system.

1. Research objective and system boundary

The source research proposal WP-012 asks whether a recurring mathematical kernel in a multi-step AI workflow can be reused and audited while preserving domain-specific accuracy. The demonstrator answers that question for a controlled, fixed model by separating execution from orchestration, with no runtime performance comparison.

Problem setting

An LLM can suggest a calculation, but a reproducible tool must be able to identify exactly which mathematical expressions, numerical inputs and output values were used. The host validates a structured request independently of any model-generated narrative. A persistent paceval runtime evaluates only approved graph objects; the trace can be replayed by a separate auditor.

Four explicit paceval computations

Stage	Expression	Role
SQUARE	t1 = x*x	Reusable preprocessing
LOG1P	t2 = ln(1+t1)	Logarithmic transform
DENOM	t3 = 1+a*x	Ratio denominator
RATIO	z = t2/t3	Final result, v1.0

Version 1.1 is a distinct reviewed model, not an equivalent rewriting: it replaces the final expression with **z = t2/t3 + 0.05*x**; the first three graphs remain unchanged. Variable declarations passed to each paceval graph are the fixed ordered signature `x a t1 t2 t3`.

Responsibility separation

paceval: graph creation, scalar and batched mathematical evaluation, deletion, optional TINC diagnostics. **C++ host:** formula registration, request schema, variable bounds, dependencies, version management, native sanity checks and audit logging. **Independent Python:** 100-digit audit of all stages, formula hashes and record hashes.

2. Execution architecture and official SDK

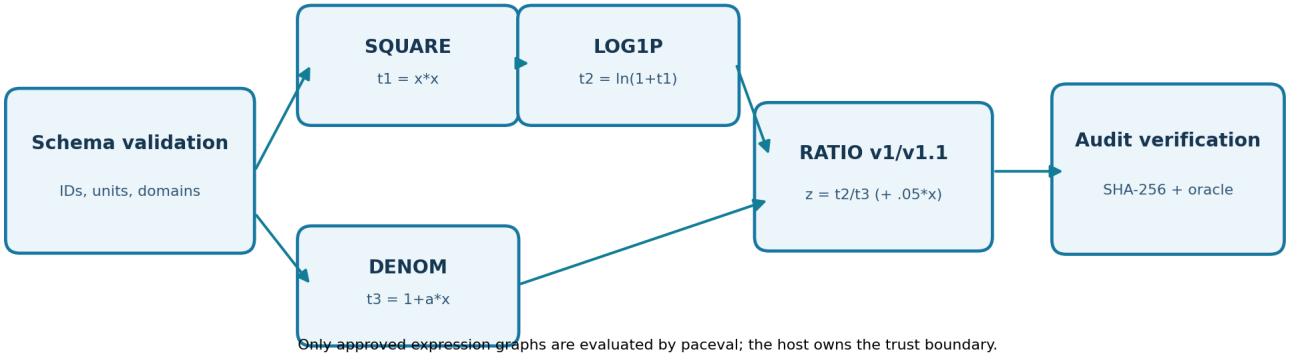


Figure 1. The validated request feeds four immutable mathematical graph nodes; outputs are passed between stages by the host.

Direct SDK integration

The application includes `paceval_main.h` and compiles `paceval_main.cpp` as part of its CMake target. CMake links the separately supplied Linux shared runtime. The host calls `paceval_InitializeLibrary`, `paceval_CreateComputation`, `paceval_dGetComputationResultExt` and `paceval_DeleteComputation`. An actual non-error batch return is `false`; per-result error codes and finite values are additionally checked.

Explicit dependency order

The first graph computes t1 for a batch of accepted requests; the second uses the returned t1 values; the third computes t3; the fourth uses t2 and t3, selecting RATIO v1.0 or v1.1. Data are transferred as double-precision scalars in the declared variable order. The same five computation objects persist during the run.

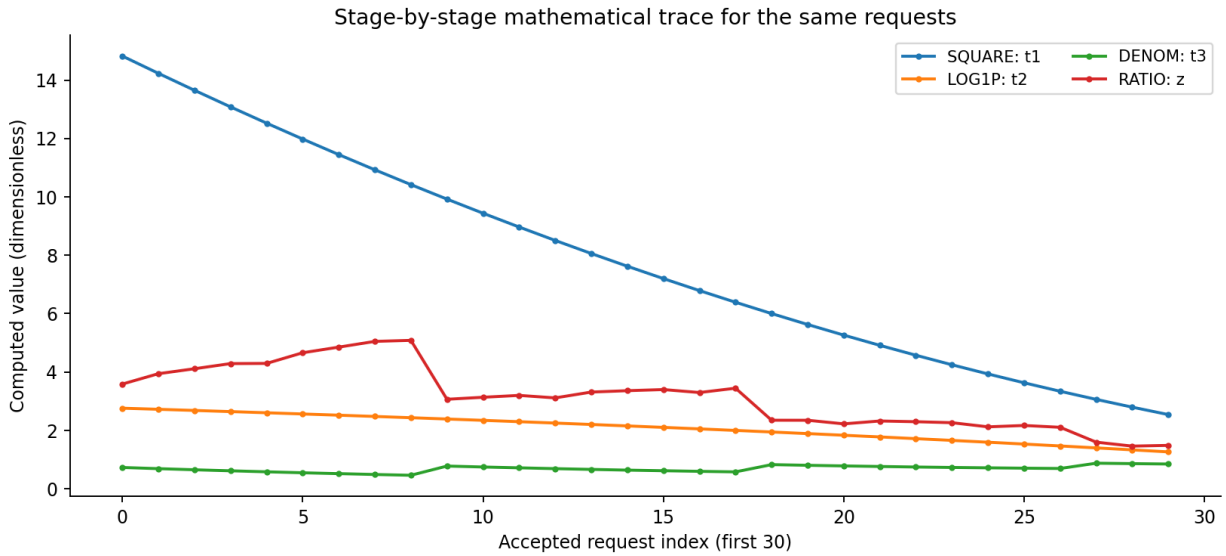


Figure 2. Recorded intermediate values for the first 30 accepted requests, based on the published steps.csv file.

3. Structured request contract and rejection policy

The local JSONL request adapter requires `request_id, formula_id, version, x, a, unit`; extra fields are rejected. It passes typed values to a persistent C++ process through a strictly delimited local protocol. The C++ host validates again: adapter validation alone is not trusted.

Control	Implemented rule
Formula and version	CHAIN_LOGRATIO; version 1.0 or 1.1 only
Input domain	Finite binary64; abs(x) <= 4; abs(a) <= 2
Denominator	abs(1+a*x) >= 0.05 before mathematical execution
Units	x, a and z are dimensionless in this synthetic model
Identifiers	1–48 characters, ASCII letters/digits/_/-; duplicates rejected
Unexpected fields	Reject invalid field counts, unknown models, malformed numbers and nonfinite values

Application controls are not paceval engine limits. The API supports significantly larger expressions and variable sets subject to the supported operations and resources. The narrower limits here are intentional safeguards for untrusted requests.

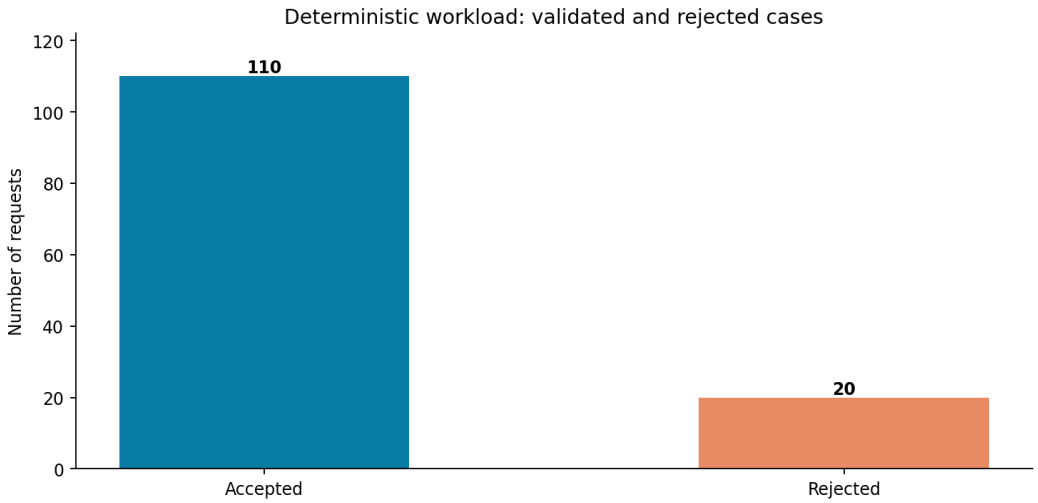


Figure 3. One hundred initial valid tasks plus ten repeated-mathematics tasks; twenty intentionally invalid cases.

4. Immutable formula IDs and audit-chain design

Each node identity includes its symbolic name, version, expression text and variable signature. SHA-256 is calculated over the canonical serialization `node|version|expression|variables`. Formula hashes in the recorded step trace must agree with the five registered model entries.

A successful request generates six records: INPUT, four STEP records and RESULT. A rejected request generates one REJECT record. The original request payload is committed by a SHA-256 digest. Each audit record contains a sequence number, request ID, record type, stage number, node, model version, formula digest, value or input digest, status, reason, predecessor digest and current digest.

Digest chain

Let H0 be 64 hexadecimal zeros and Cn be a fixed-order, pipe-delimited canonical record payload. For each event, **Hn = SHA256(H(n-1) | "|" | Cn)**. The terminal Hn and the record count are saved separately as an audit-head manifest. The verifier checks all 680 records, formula hashes, mathematical stages and input commitments.

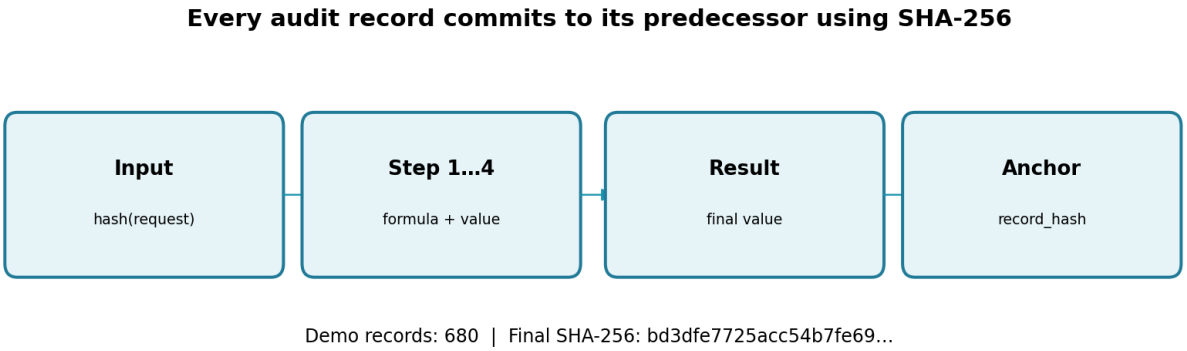


Figure 4. Reproducible record chaining. Tamper evidence requires the terminal digest to be anchored independently.

Evidence boundary

A plain hash chain is not an authenticated record store. A privileged attacker could rewrite both the records and their untrusted terminal digest. A production system would need an independently retained signed or anchored head, authorization, durable storage, privacy protection and secure process identity.

5. Independent numerical results

Observed output	Verified result
Valid requests	110/110 executed; 440 stage values recorded
Invalid requests	20/20 rejected with an explicit reason
High-precision audit	440/440 stages within fixed relative/absolute tolerance
Maximum absolute discrepancy	8.7929664 x 10^-16 (stage values, tested inputs)
SHA-256 trace	680/680 records plus independent head digest verified
Negative tampering test	Edited audit record detected by independent verifier
Adapter examples	12 accepted; 4 rejected; persistent graph reuse verified

Precision interpretation

Python mpmath at 100 decimal digits recomputes the mathematical stages from the exact binary64 values handed to the C++ host. The reference does not simply parse human-readable numbers as unrelated exact decimals. The reported maximum error is for the selected input domain and this five-graph catalog; it is not a global error bound.

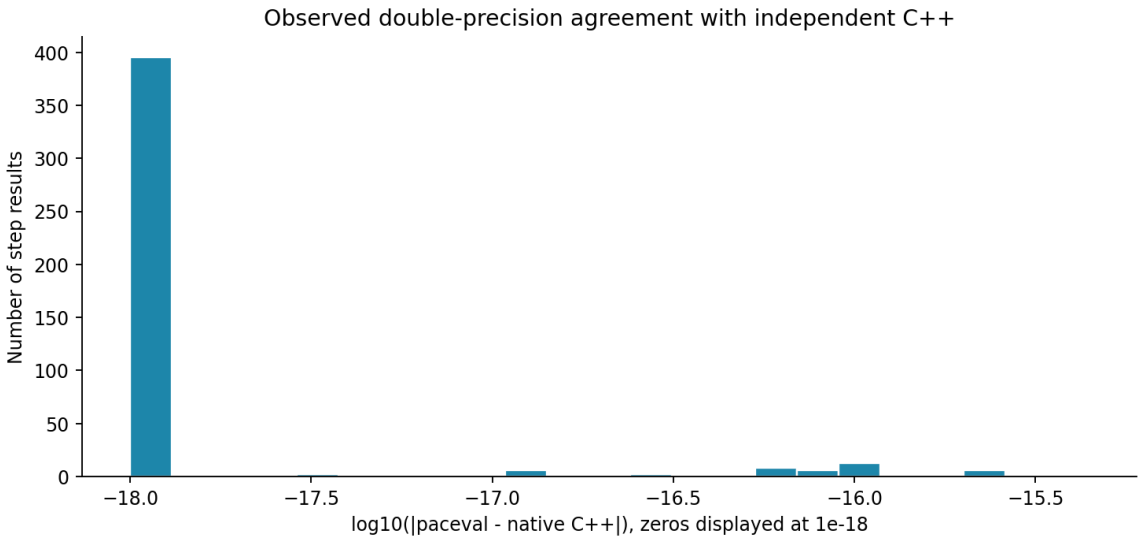


Figure 5. Differences between paceval and a separately implemented native C++ reference over 440 recorded steps.

6. Model updates and error screening

The v1.1 output adds 0.05*x to the v1.0 expression while retaining the upstream SQUARE, LOG1P and DENOM graph objects. Sixteen paired inputs were calculated with both final-node versions. The independently verified difference equals 0.05*x within numerical tolerance. Earlier models are not overwritten when the new version is registered.

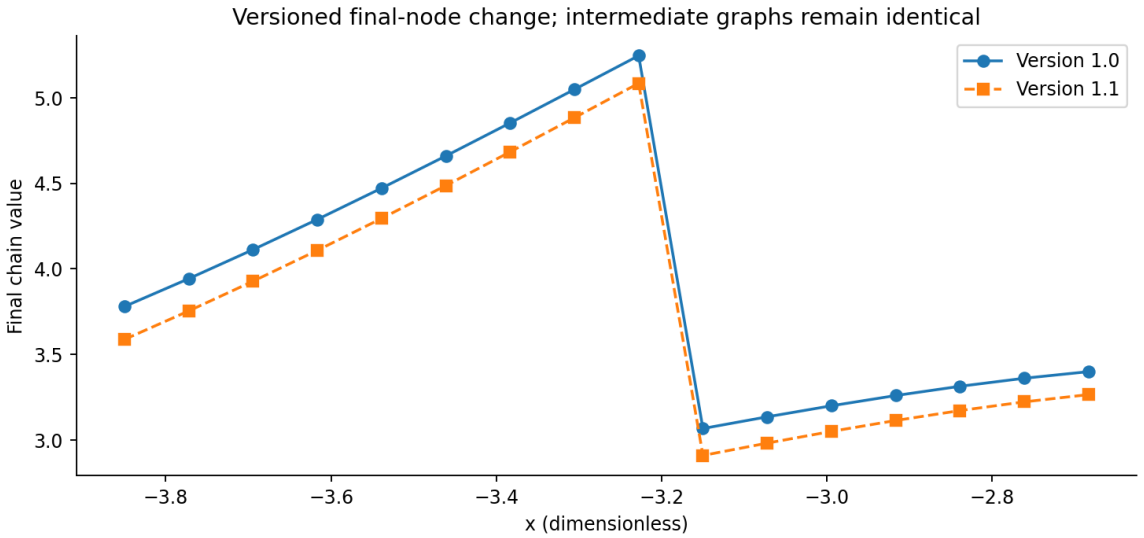


Figure 6. Both independently versioned final expressions evaluated on the same sixteen input pairs.

Deliberately faulty request classes

The test pool includes unknown formula IDs, unknown versions, incompatible units, out-of-domain magnitudes, a zero denominator, duplicate request IDs, missing numeric fields, NaN and extra fields. A rejected request has an audit record but no paceval stage execution.

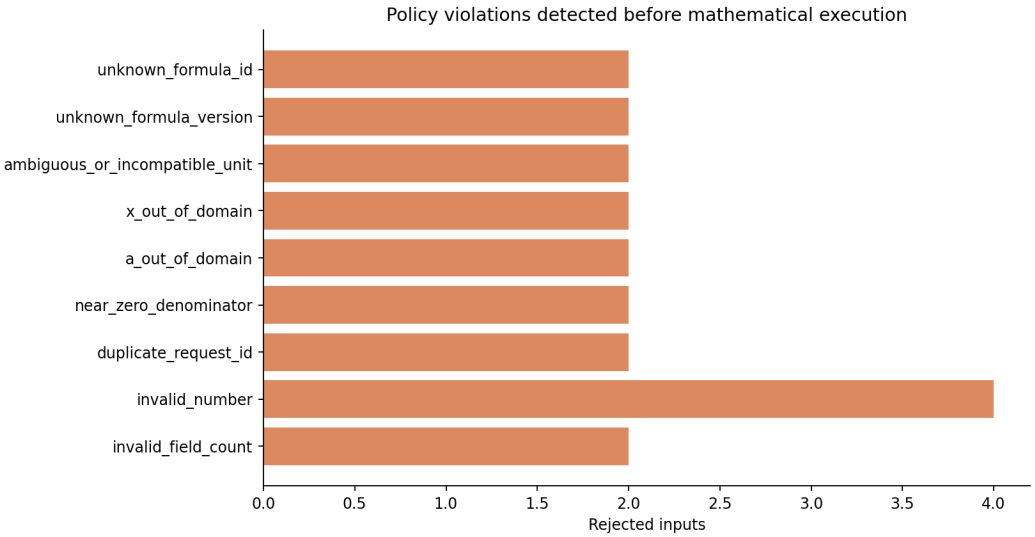


Figure 7. Deterministically generated negative cases and their explicit rejection categories.

7. TINC exploratory diagnostic and numerical boundaries

A separate TINC-enabled graph evaluates **ln(1+x*x)** at 24 deterministic points, independent of the 440 ordinary double-precision stage checks. The returned raw interval endpoints are compared to an exact-input 100-digit mpmath reference.

Observation	Outcome
Raw sample intervals enclosing the exact reference	23 of 24
Missed sample index	12
Input for missed sample	x = 2.2204460492503131e-16
Returned interval	[0, 0]
Correct real-valued reference	Approximately 4.9303806576313e-32

This near-zero case is scientifically informative: a point interval at zero does not enclose a very small positive exact real result. The diagnostic is preserved and displayed, not relabeled as successful. Consequently this demonstration makes no universal or certified interval-enclosure claim. The main arithmetic chain is independently checked on all accepted requests.

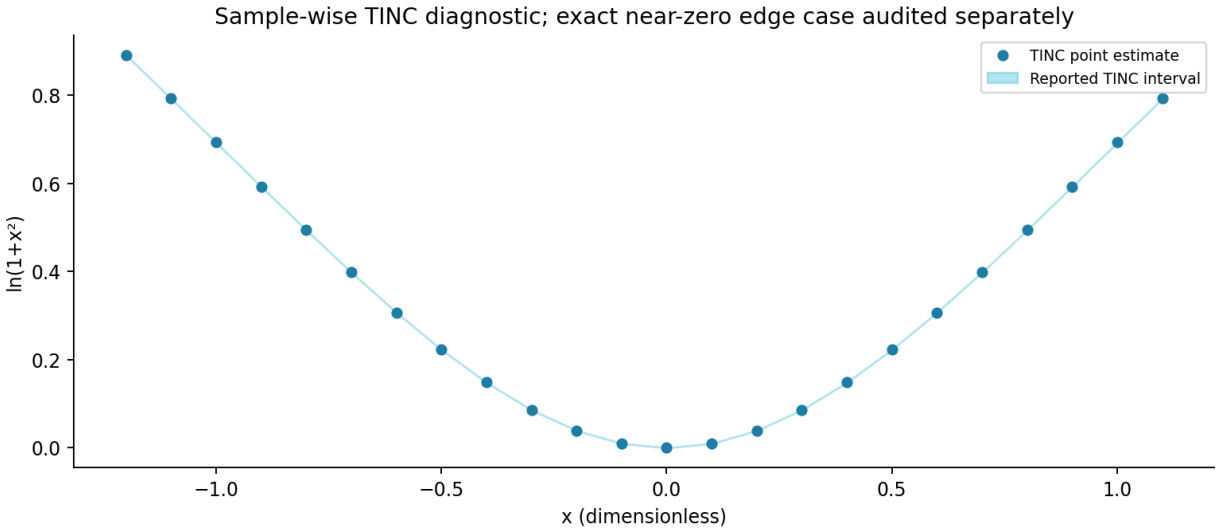


Figure 8. Recorded TINC estimates and raw intervals. At chart scale, the near-zero non-inclusion is visually imperceptible and is therefore disclosed numerically in the table.

8. Reproduction, limitations and research continuation

Reproducibility setup

Component	Reproduced environment / responsibility
Runtime	Original supplied paceval Linux x86-64 shared library, not repackaged
C++	C++17, GCC / Clang, official SDK source units
Hashing	OpenSSL libcrypto / SHA-256 with independent Python hashlib audit
Reference	Python 3 + mpmath at 100 decimal digits
Commands	cmake -S src -B build -DPACEVAL_RUNTIME=/abs/path/to/runtime.so; cmake --build build; ctest --test-dir build --output-on-failure
Included examples	requests.csv; models.csv; steps.csv; results.csv; audit_steps.csv; audit_head.txt; TINC and agent JSONL samples

All source, example and figure references resolve relative to the repository root. The standalone adapter is JSONL, not MCP. The demo has no LLM provider, no claims about causal appropriateness of AI tool decisions and no production-level identity or audit authentication. Handling of numerical singularities is an explicit host policy.

Selected technical references

1. paceval C/C++ API, reusable computations and batch evaluation
https://paceval.com/api/

2. Model Context Protocol, 2025-11-25, server tools
https://modelcontextprotocol.io/specification/2025-11-25/server/tools

3. OWASP, MCP Security Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/MCP_Security_Cheat_Sheet.html

4. NIST, AI Risk Management Framework
https://www.nist.gov/itl/ai-risk-management-framework

Research conclusion

WP-012 successfully demonstrates an independently verified, versioned and auditable mathematical execution layer for controlled multi-step AI-tool-like requests. The proven scope is the synthetic fixed formula catalog, a working direct paceval SDK integration, and an independently replayable local audit trail. A live MCP transport and separately anchored, authenticated logs are meaningful next engineering steps.